Algoritmo e Laboratório de Programação II

Aula 8 – Ponteiros - parte 4 (Nível Avançado)

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Área de alocação dinâmica: heap

- Suponhamos que você vá escrever um programa interativo e não conheça de antemão quantas entradas de dados serão fornecidas.
- O cadastro de livros de uma biblioteca é um bom exemplo.
- Você pode reservar uma quantidade de memória que pensa ser razoável, como, por exemplo, declarar uma matriz para armazenar 50 estruturas do tipo Livro.

Área de alocação dinâmica: heap

- Nesse caso, o compilador aloca memória para armazenar toda a matriz, e isso se torna ineficiente caso não ocupemos todo o espaço reservado.
- Por outro lado, poderia ser necessário armazenar mais de 50 livros, e a matriz não atenderá a esse requisito.
- A solução para esse tipo de problema é solicitar memória toda vez que se fizer necessário.

Área de alocação dinâmica: heap

- O mecanismo para aquisição de memória em tempo de execução se dá por meio da função de biblioteca-padrão **malloc()**.

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

A Função malloc()

- A função malloc() recebe como argumento um número inteiro positivo que representa a quantidade de bytes de memória desejada.
- Solicita memória ao sistema operacional e retorna um ponteiro void para o primeiro byte do novo bloco de memória que foi alocado.

A Função malloc()

- Se não houver memória suficiente para satisfazer a exigência, malloc() retornará um ponteiro com o valor NULL.
- Os bytes alocados são inicializados com lixo.
- O fragmento de programa a seguir aloca memória para uma variável estrutura do tipo data.

```
struct Data *ptr;
ptr = (struct Data *)(malloc(sizeof(struct Data)));
```

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

A Função calloc()

- A função calloc() aloca uma matriz de elementos inicializados com zero (0).
- Internamente, a função calloc() chama a função malloc().
- A nova função recebe dois números inteiros como argumentos:
  - O primeiro indica o número de itens desejados
  - O segundo o tamanho de cada item.

A Função calloc()

- Então, retorna um ponteiro void apontando para o primeiro byte do bloco solicitado.
- O fragmento de programa a seguir aloca memória para uma matriz de 100 inteiros:

```
int *memnova;
memnova = (int *) calloc(100, sizeof(int));
```

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

A Função free()

- Uma vez alocada memória dinamicamente, ela continuará ocupada até que seja desalocada explicitamente pela função free().
- Em outras palavras, uma variável criada com a função malloc() ou calloc() existirá e poderá ser acessada em qualquer parte do programa enquanto não for liberada por meio da função free() e seu espaço de memória devolvido ao sistema operacional.

A Função free()

- A função free() recebe como argumento um ponteiro para uma área de memória previamente alocada por malloc() ou calloc(), e então libera essa área para uma possível utilização futura.

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Alocação de Tipos Básicos Usando Memória Dinâmica

- O nosso primeiro exemplo mostra como criar uma variável dinamicamente.

Prog29.c

```
1  /* Prog29.c */
2  /* mostra o uso de malloc() */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      int *pi;
9
10     pi = (int *) malloc(sizeof(int));
11
12     puts("Digite um número: ");
13     scanf("%d", pi);
14     printf("\nVocê digitou o número %d\n", *pi);
15
16     free(pi);
17
18     system("PAUSE");
19     return 0;
20 }
```

Dimensionamento de Matrizes em Tempo de Execução

- A determinação do tamanho de uma matriz pode ser feita em tempo de execução.
- Vamos modificar o programa que calcula média para que o usuário indique quantas notas serão inseridas.

Dimensionamento de Matrizes em Tempo de Execução

Prog30.c

```c
/* Prog30.c */
/* Alocação dinâmica da matriz com calloc() */
#include <stdio.h>
#include <stdlib.h>

float media(float *, int );/*protótipo */

int main()
{
    float *notas , m;
    int tamanho, i;

    puts("Qual é o número de notas? ");
    scanf("%d", &tamanho);

    notas = (float *)calloc(tamanho,sizeof(float));

    for(i=0; i < tamanho; i++)
    {
        printf("Digite a nota do aluno %d: ", i + 1);
        scanf("%f", notas+i);
    }
```

Dimensionamento de Matrizes em Tempo de Execução

```
24      m = media( notas, tamanho );
25      printf("Média das notas: %.2f\n", m);
26
27      free(notas);
28      notas = NULL;
29
30      system("PAUSE");
31      return 0;
32  }
33
34  /* Calcula a média dos valores da matriz */
35  float media(float *lista, int tamanho)
36  {
37      int i;
38      float m=0.0;
39      for(i=0; i < tamanho ; i++) m += *(lista++);
40      return  m/tamanho;
41  }
```

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Criando uma Lista Ligada

- Lista ligada é um algoritmo de armazenamento de dados que muitas vezes supera o uso de uma matriz ou de uma matriz de ponteiros.
- A lista ligada assemelha-se a uma corrente em que os registros de dados estão pendurados sequencialmente.
- O espaço de memória para cada registro é obtido pela função malloc(), conforme sugere a necessidade de adicionar itens à lista.

Criando uma Lista Ligada

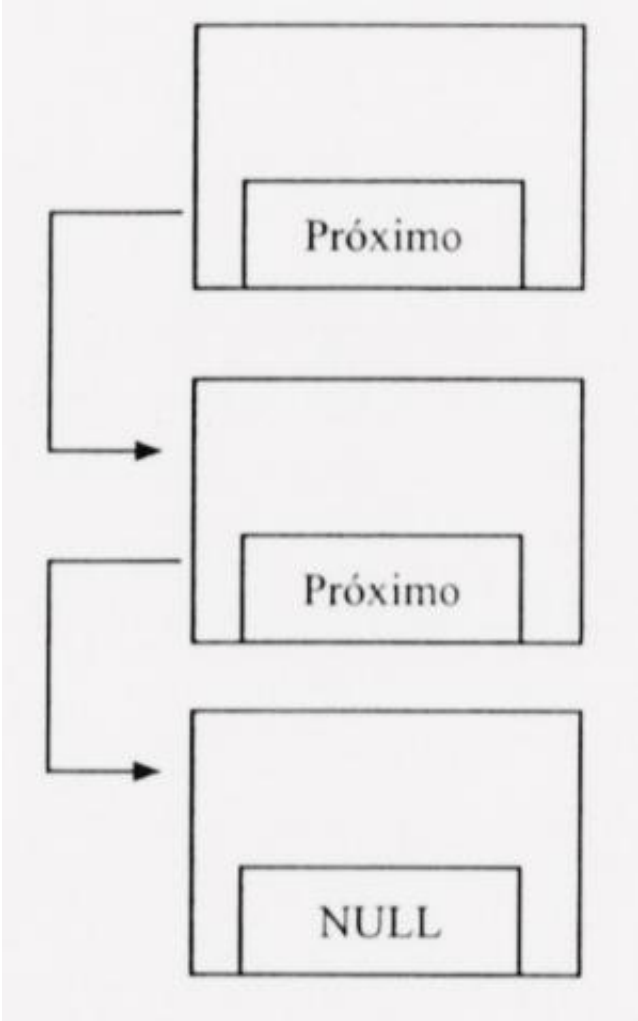
- Cada registro é conectado ao próximo por meio de um ponteiro.
- O último registro contém um ponteiro com o valor NULL, e cada registro anterior contém um ponteiro apontando para o próximo.
- Cada registro é representado por uma variável estrutura do tipo Livro.

Criando uma Lista Ligada

- Cada estrutura contém um conjunto de membros para armazenar os dados de um livro: título, autor, número do registro e preço.
- Há um membro a mais, um ponteiro, para armazenar o endereço do próximo registro.
- A lista, como um todo, é acessada por meio de um ponteiro para a primeira estrutura, chamado cabeça.

Criando uma Lista Ligada

- Esquema de uma lista ligada:



O Programa

Prog31.c

```
1   /* Prog30.c */
2   /* Mostra a implementação de uma lista ligada */
3   #include <stdio.h>
4   #include <stdlib.h>
5   #include <conio.h> /* para getch() */
6
7   typedef struct Livro
8   {
9       char Titulo[30];
10      char Autor[30];
11      int  NumReg;
12      double Preco;
13      struct Livro *Proximo;
14  } Livro;
15
16  Livro *primeiro, *atual, *NovoLivro;
```

O Programa

```
18  void GetLivro()
19  {
20      char temp[80];
21      NovoLivro = (Livro *)malloc(sizeof(Livro));
22      if( primeiro == (Livro *)NULL)
23          primeiro = atual = NovoLivro;
24      else
25      {
26          atual = primeiro;
27          while( atual->Proximo != (Livro *)NULL)
28              atual = atual->Proximo;/* procura novo item */
29          atual->Proximo = NovoLivro;
30          atual = NovoLivro;
31      }
32      printf("Digite titulo: ");
33      gets(atual->Titulo);
34      printf("Digite autor: ");
35      gets(atual->Autor);
36      printf("Digite o número do registro: ");
37      gets(temp);
38      atual->NumReg = atoi(temp);
39      printf("Digite o preço: ");
40      gets(temp);
41      atual->Preco = atof(temp);
42      atual->Proximo=(Livro *)NULL;
43  }
```

O Programa

```
45  void PrintLivro()
46  {
47      if( primeiro == (Livro *)NULL)
48      {
49          puts("Lista vazia");
50          return;
51      }
52
53      atual = primeiro;
54
55      do
56      {
57          printf( "Titulo: %s\n", atual->Titulo);
58          printf( "Autor : %s\n" , atual->Autor);
59          printf( "No.Reg: %d\n" , atual->NumReg);
60          printf( "Preco : %.2f\n\n" , atual->Preco);
61          atual = atual->Proximo;
62      }
63      while(atual != NULL);
64  }
```

O Programa

```
66  int main()
67  {
68      char ch;
69      primeiro = (Livro *) NULL; /* sem dados ainda */
70
71      do
72      {
73          GetLivro();
74          puts("\nInserir outro livro (s/n)? ");
75          ch = getch();
76      }
77      while( (ch != 'n') && (ch != 'N'));
78
79      puts("\nLISTA DOS LIVROS CADASTRADOS");
80      puts("============================");
81
82      PrintLivro();
83
84      system("PAUSE");
85      return 0;
86  }
```

O Programa

- Se você observou bem, a ideia básica é que cada variável estrutura contenha um ponteiro para a próxima estrutura da lista; e que o ponteiro da última estrutura contenha um ponteiro nulo (NULL).
- O ponteiro primeiro será usado para armazenar o endereço da primeira estrutura da lista.
- Esse é um endereço “chave”, visto que indica onde encontrar a lista.

O Programa

- O ponteiro é inicializado com NULL por ser externo.
- NULL é uma constante definida no arquivo stdio.h como o valor zero.
- A linguagem C garante que um ponteiro que aponta para algum endereço válido nunca terá o valor zero, de forma que esse valor pode ser retornado para sinalizar um evento anormal, nesse caso, o fim da lista.

O Programa

- Escrevemos NULL em vez de zero para indicar mais claramente que esse é um valor especial para um ponteiro.
- Como ele é inteiro e será atribuído a um ponteiro, devemos usar o operador de conversão de tipo (Livro*).

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Adicionando um Livro à Lista

- A função Getivro() adiciona um livro à lista na instrução:

```
NovoLivro = (Livro *)malloc(sizeof(Livro));
```

- Em seguida, a função verifica se esse é o primeiro item a ser colocado na lista:

```
if( primeiro == (Livro *)NULL)
    primeiro = atual = NovoLivro;
```

- Em caso positivo, o novo endereço é atribuído aos ponteiros primeiro e a atual.

Adicionando um Livro à Lista

- Cada membro individual é acessado por meio do operador ->.
- Finalmente, é atribuido NULL ao membro Proximo.
- Entretanto, se esse não for o primeiro item da lista, o programa usa um laço while para encontrar o fim da lista.

Adicionando um Livro à Lista

- Começa atribuindo primeiro a atual pela expressão: `atual = primeiro;`
- E em seguida, o laço while e verifica se o ponteiro da estrutura atual é NULL. Se não for, o laço while atribui o endereço em atual->Proximo a atual e volta ao teste.

```
while( atual->Proximo != (Livro *)NULL)
    atual = atual->Proximo;/* procura novo item */
```

- O ciclo termina quando é encontrada a última estrutura.

Adicionando um Livro à Lista

- Ao seu membro ponteiro, atribuímos o endereço da nova estrutura.

```
atual->Proximo = NovoLivro;
atual = NovoLivro;
```

- Em seguida, os membros da nova estrutura são preenchidos com os dados digitados pelo usuário, com exceção do membro ponteiro.
- A ele atribuímos NULL.

```
atual->Proximo=(Livro *)NULL;
```

- O novo livro é inserido no final da lista.

Imprimindo os Dados da Lista

- Para imprimir os dados dos livros cadastrados devemos seguir a cadeia de ponteiros, da primeira estrutura para a próxima, até encontrar NULL.
- A função PrintLivro() inicia verificando se a lista está vazia.
- Se não estiver, atribui o valor do ponteiro primeiro ao ponteiro atual, que irá varrer a lista.

```
atual = primeiro;
```

Imprimindo os Dados da Lista

- Em seguida, entra no laço e imprime os valores dos membros da estrutura apontada por atual; em seguida, acerta o ponteiro atual para que aponte para a próxima estrutura:

```
atual = atual->Proximo;
```

- O laço termina quando a última estrutura for atingida.

Implementações Adicionais

- Podem ser feitos vários refinamentos no uso de listas encadeadas, como guardar o endereço do último item da lista no primeiro.
- Dessa maneira, se quisermos adicionar um novo item na lista, o programa poderá ir diretamente ao último item.
- Igualmente, a lista pode ser encadeada para trás ou para a frente (como mostramos), ou ainda de modo circular.

Implementações Adicionais

- Eliminar um item da lista é, razoavelmente, fácil.
- Se o registro a res eliminado for B, então o ponteiro da estrutura anterior A será alterado para que aponte para a estrutura C.
- Se for desejado liberar a memória usada por estruturas apagadas, a função de biblioteca free() deverá ser usada.

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Argumentos da Linha de Comando

- Se você já trabalhou com os sistemas operacionais Unix, Linux ou DOS, com certeza já deve estar acostumado a usar argumentos da linha de comando.
- Por exemplo: `C:\>FORMAT D: /s /u`
- Nesse exemplo, D: é um argumento da linha de comando, /s e /u são outros dois .
- Os itens digitados na linha de comando do DOS são chamados argumentos da linha de comando.

Argumentos da Linha de Comando

- Usuários do Windows podem entrar argumentos da linha de comando após o nome do programa na opção Executar do menu Iniciar.
- Como podemos escrever programas em C que acessem esses argumentos?
- Os argumentos digitados na linha de comando são enviados pelo sistema operacional como argumentos da função main().

Argumentos da Linha de Comando

- Para que essa função possa reconhecê-los, é necessário declará-los como é feito em qualquer função C.
- Eis um exemplo:

Argumentos da Linha de Comando

Prog32.c

```c
/* Prog32.c */
/* Mostra argumentos da linha de comando */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) /* int main(int argc, char *argv[]) */
{
    int i;

    printf("Número de argumentos: %d\n", argc);

    for(i=0; i<argc; i++)
        printf("Argumento número %d: %s\n", i, argv[i]);

    system("PAUSE");
    return 0;
}
```

Argumentos da Linha de Comando

- A função main() recebe dois argumentos: argc e argv.

```
int main(int argc, char **argv) /* int main(int argc, char *argv[]) */
```

- O primeiro argumento representa o número de argumentos digitados na linha de comando, neste caso sete.
- O segundo argumento corresponde a uma matriz de ponteiros para strings, as quais representam argumentos da linha de comando.

Argumentos da Linha de Comando

- As cadeias de caracteres podem ser acessadas por meio da notção matriz, argv[0], argv[1] etc. ou por meio da notação ponteiro, *(argv + 0), *(argv + 1) etc.
- A primeira cadeia, argv[0], é sempre o nome do programa sendo executado e o seu caminho de localização no disco.
- O sistema operacional entende o espaço em branco como separador dos argumentos.

Argumentos da Linha de Comando

- Os nomes argc (ARGument Count) e argv (ARGument Values) são tradicionalmente usados para esse fim, mas qualquer outro nome poderia ser usado por você.

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Atribuindo Múltiplos Argumentos e Teclas de Função

- O proximo exemplo, usa um comando do driver Ansi.sys para programar uma tecla de função.
- Para executar o programa, o driver Ansi.sys deve estar instalado e rodando no prompt ativo do DOS. Siga os passos da aula correspondente no curso de C básico e Intermediário.

Atribuindo Múltiplos Argumentos e Teclas de Função

- Embora o que você digite na linha de comando tenha a aparência de uma cadeia de caracteres, o sistema operacional interpreta-o como uma série de variáveis separadas por espaços em branco.
- Para atribuir uma cadeia de caracteres de múltiplas palavras para uma tecla de função, é necessário combinar os diversos argumentos da linha de comando em uma única matriz.

Atribuindo Múltiplos Argumentos e Teclas de Função

- O programa seguinte faz isso usando a função strcpy() para colocar o primeiro argumento na matriz vazia, e então usa strcat() para concatenar cada um dos argumentos seguintes, colocando um espaço entre eles.

Atribuindo Múltiplos Argumentos e Teclas de Função

Prog33.c

```
1  /* Prog33.c */
2  /* Atribui o texto digitado na linha de comando a uma tecla de função */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  int main(int argc, char **argv)
8  {
9      int tecla, j;
10     char str[80];
11
12     if(argc < 3)
13     {
14         printf("Exemplo de uso: C:\\>Teclaf 2 dir *.c");
15         exit(1) ;
16     }
17     tecla = atoi(argv[1]);
18     strcpy(str,argv[2]);
```

Atribuindo Múltiplos Argumentos e Teclas de Função

```
19      for(j=3; j< argc; j++)
20      {
21          strcat(str," ");
22          strcat(str,argv[j]);
23      }
24      if( strcmp(str, "null" )==0) /* limpa tecla */
25          strcpy(str,"");
26      printf("\x1B[0;%d;\"%s\";13p",tecla+58,str);
27      system("PAUSE");
28      return 0;
29  }
```

Atribuindo Múltiplos Argumentos e Teclas de Função

- Vamos ver o que ocorre quando executamos o programa no prompt de comando da console, digitando o seguinte texto:

```
C:\>MeusProjetos\>Prog33 2 dir *.*
```

- Após a execução, pressione a tecla F2 e você verá impressos os nomes dos arquivos gravados na pasta.
- Se você quiser limpar o que foi atribuído à tecla de função 2, digite a seguinte instrução:

```
C:\>MeusProjetos\>Prog33 2 null
```

Roteiro

- Área de alocação dinâmica: heap
- Alocando e desalocando memória do heap
- A função malloc()
- A função calloc()
- A função free()
- Alocação de tipos básicos usando memória dinâmica
- Dimensionando matrizes em tempo de execução
- Criando uma lista ligada
- Adicionando um livro à lista
- Argumentos da linha de comando
- Atribuindo múltiplos argumentos a teclas de função
- Alocando matrizes e vetores

Alocando Matrizes e Vetores

- Veremos agora dois exemplos de como criar funções que aloquem e desaloquem matrizes e vetores na memória do computador.
- Para a alocação devemos usar a função malloc() e para liberar a memória usaremos a função free();
- SEMPRE deve-se atribuir o valor NULL para um ponteiro que não é mais utilizado, isso evita falhas graves de acesso indevido à memória em softwares grandes e complexos.

Alocando Matrizes e Vetores

Prog34.c

```c
/* Prog34.c */
/* Alocação de Vetores */
#include <stdio.h>
#include <stdlib.h>

int *aloca_array(int);              /* Função para reservar a mémória do array                    */
void init_array(int *, int);        /* Função para preencher o array com valores aleatórios       */
void print_array(int *, int);       /* Função para printar o array na tela                        */
void destruir_array(int **, int);   /* Função para liberar o espaco alocado com a função free()   */


int main()
{
    unsigned int size;
    int *Vet; /*Cria o ponteiro para o array */

    printf("Informe o número de elementos do array: ");
    scanf("%d", &size);

    Vet = (int *)aloca_array(size);

    init_array(Vet, size);
    print_array(Vet, size);

    destruir_array(&Vet, size);

    print_array(Vet, size);

    return 0;
}
```

Alocando Matrizes e Vetores

```
32  /* função reserva a memória usando a função malloc e */
33  /* testa se o ponteiro foi corretamente alocado      */
34  int *aloca_array(int Size)
35  {
36      int *Array = (int *)malloc(Size*sizeof(int));
37
38      if(Array == NULL)
39      {
40          printf("\tERRO!\n\n**Não foi Possivel Alocar\n");
41          getche();
42          exit(1);
43      }
44      return ((int *)Array);
45  }
46
47  void init_array(int *Array, int Size)
48  {
49      unsigned register int i = 0;
50      for(i = 0; i < Size; i++)
51      {
52          Array[i] = rand() % Size;
53      }
54  }
```

Alocando Matrizes e Vetores

```
56  /* função testa se o ponteiro existe antes de imprimir */
57  void print_array(int *Array, int Size)
58  {
59      unsigned register int i = 0;
60
61      if(Array == NULL)
62      {
63          printf("\tERRO!\n\n**Não foi Possivel Imprimir: Ponteiro NULL\n");
64          return;
65      }
66
67      for(i = 0; i < Size; i++)
68      {
69          printf("%d \t", Array[i]);
70          printf("\n");
71      }
72      printf("\n");
73  }
74  /* função testa se o ponteiro existe antes de destruir */
75  /* destroi ponteiro chamando a função free() e depois  */
76  /* coloca o ponteiro como NULL para evitar que seja    */
77  /* usado em outras partes do código                    */
78  void destruir_array(int **Array, int Size)
79  {
80      if(Array == NULL)
81      {
82          printf("\tERRO!\n\n**Não foi Possivel Destruir: Ponteiro NULL\n");
83          return;
84      }
85
86      free(*Array);
87      *Array = NULL;
88  }
```

Alocando Matrizes e Vetores

- Note como a função destruir_array() funciona;
- O ponteiro quando passado para uma função é passado por valor, ou seja, na função é gerada uma cópia do ponteiro.
- Sendo assim devemos passar para a função o endereço da variável ponteiro e receber na função um ponteiro de ponteiro para alterar o ponteiro original.

Alocando Matrizes e Vetores

Prog35.c

```c
/* Prog35.c */
/* Alocação de Matrizes */
#include <stdio.h>
#include <stdlib.h>

int **aloca_array(int, int);              /* Função para reservar a mémória do array
void init_array(int **, int, int);        /* Função para preencher o array com valores aleatórios
void print_array(int **, int, int);       /* Função para printar o array na tela
void destruir_array(int ***, int, int);   /* Função para liberar o espaço alocado com a função free()


int main()
{
    unsigned int Lin, Col;
    int **Mat; /*Cria o ponteiro para o array */

    printf("\nInforme o número de linhas do array: ");
    scanf("%d", &Lin);
    printf("\nInforme o número de colunas do array: ");
    scanf("%d", &Col);

    Mat = (int **)aloca_array(Lin, Col);

    init_array(Mat, Lin, Col);

    print_array(Mat, Lin, Col);

    destruir_array(&Mat, Lin, Col);

    print_array(Mat, Lin, Col);

    return 0;
}
```

Alocando Matrizes e Vetores

```
35  /* função reserva a memória usando a função malloc e */
36  /* testa se o ponteiro foi corretamente alocado      */
37  /* deve-se reservar um vetor de ponteiros para alocar as linhas */
38  /* para cada linha se aloca um vetor com os elementos das colunas */
39  int **aloca_array(int lin, int col)
40  {
41      unsigned register int i;
42      int **Array = (int **)malloc(lin * sizeof(int*));
43
44      if(Array == NULL)
45      {
46          printf("\tERRO!\n\n**Não foi Possivel Alocar\n");
47          getche();
48          exit(1);
49      }
50      for(i = 0; i < lin; i++)
51      {
52          Array[i] = (int *)malloc(col * sizeof(int));
53          if(Array[i] == NULL)
54          {
55              printf("\tERRO!\n\n**Não foi Possivel Alocar\n");
56              getche();
57              exit(1);
58          }
59      }
60      return ((int **)Array);
61  }
```

Alocando Matrizes e Vetores

```
63  void init_array(int **Array, int lin, int col)
64  {
65      unsigned register int i = 0, j = 0;
66
67      for(i = 0; i < lin; i++)
68      {
69          for(j = 0; j < col; j++)
70          {
71              Array[i][j] = rand() % (lin * col);
72          }
73      }
74  }
75
76  /* função testa se o ponteiro existe antes de imprimir */
77  void print_array(int **Array, int lin, int col)
78  {
79      unsigned register int i = 0, j = 0;
80
81      if(Array == NULL)
82      {
83          printf("\tERRO!\n\n**Não foi Possivel Imprimir: Ponteiro NULL\n");
84          return;
85      }
86
87      for(i = 0; i < lin; i++)
88      {
89          for(j = 0; j < col; j++)
90          {
91              printf("%d \t", Array[i][j]);
92          }
93          printf("\n");
94      }
95  }
```

Alocando Matrizes e Vetores

```
 97  /* função testa se o ponteiro existe antes de destruir */
 98  /* destroi ponteiro chamando a função free() e depois  */
 99  /* coloca o ponteiro como NULL para evitar que seja    */
100  /* usado em outras partes do código                    */
101  void destruir_array(int ***Array, int lin, int col)
102  {
103      unsigned register int i = 0;
104
105      if(Array == NULL)
106      {
107          printf("\tERRO!\n\n**Não foi Possivel Destruir: Ponteiro NULL\n");
108          return;
109      }
110
111      for(i = 0; i < lin; i++)
112      {
113          free((*Array)[i]);
114      }
115      free(*Array);
116      *Array = NULL;
117  }
```

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;